

COMP2038

ALGORITHMS, CORRECTNESS AND EFFICIENCY

Analysis of Algorithms

Algorithm

- An algorithm is a **finite** sequence of computational **steps** that produces the desired **output** when applied on a certain problem with necessary **input** values.
- An algorithm is defined as an ultimate group of instructions to achieve a given set of goals to fulfill the given criteria.

Why Study Algorithms?

- Importance in Computer Science
- Real-world Applications

Algorithm Analysis

- Measuring Efficiency
 - Time Complexity
 - Space Complexity
- Best, Average and Worst Case
- Asymptotic Analysis

Goals of Analysis

- Algorithm analysis provides a theoretical foundation for the systematic evaluation of algorithmic solutions. We aim to achieve several critical objectives by analyzing the algorithms:
 - Understanding Efficiency
 - Comparison of Algorithms
 - Predict Performance
 - Guidance in Designing Algorithm
 - Hardware-Independent Evaluation

Analysis of Algorithm

```
algorithm swap (a, b)
    input: 2 integers
    output: integers being
           swabbed
    temp = a
    a = b
    b = temp
```

```
algorithm arrayMax (A, n)
    input: array A of n integers
    output: maximum element of A
    currentMax = A[0]
    for i = 1 to n-1 do
        if A[i] > currentMax then
            currentMax = A[i]
    return currentMax
```

Analysis of Algorithm

```
algorithm sum (A, n)
    input:array A of n
    integers
    output:total of n element
    in array A
    s = 0
    for i = 0 to n do
        s = s + A[i]
    return s
```

```
algorithm addition (A, B, n)
    input:array A and B of n
    integers
    output:addition of two
    matrices
    for i = 1 to n do
        for j = 1 to n do
            c[i, j] = A[i, j] + B[i, j]
```

Analysis of Algorithm

```
algorithm multiplication (A, B, n)
  input: array A and B of n integers
  output: multiplication of two matrices
  for i = 1 to n do
    for j = 1 to n do
      C[i, j] = 0
      for k = 1 to n do
        C[i, j] = C[i, j] + A[i, k] * B[k, j]
```


Time Complexities (Example)

```
for (i = 0; i < n; i ++)  
{  
    statement;  
}
```

```
for (i = 0; i < n; i ++)  
{  
    for (j = 0; j < n; j ++)  
    {  
        statement;  
    }  
}
```

Time Complexities (Example)

```
- for (i = 1; i < n; i = i * 2)
{
    statement;
}
```

```
- for (i = n; i >= 1; i = i / 2)
{
    statement;
}
```

```
- for (i = 0; i*i < n; i = i ++ )
{
    statement;
}
```

Time Complexities (Example)

```
for (i = 0; i < n; i++)  
{  
    statement;  
}
```

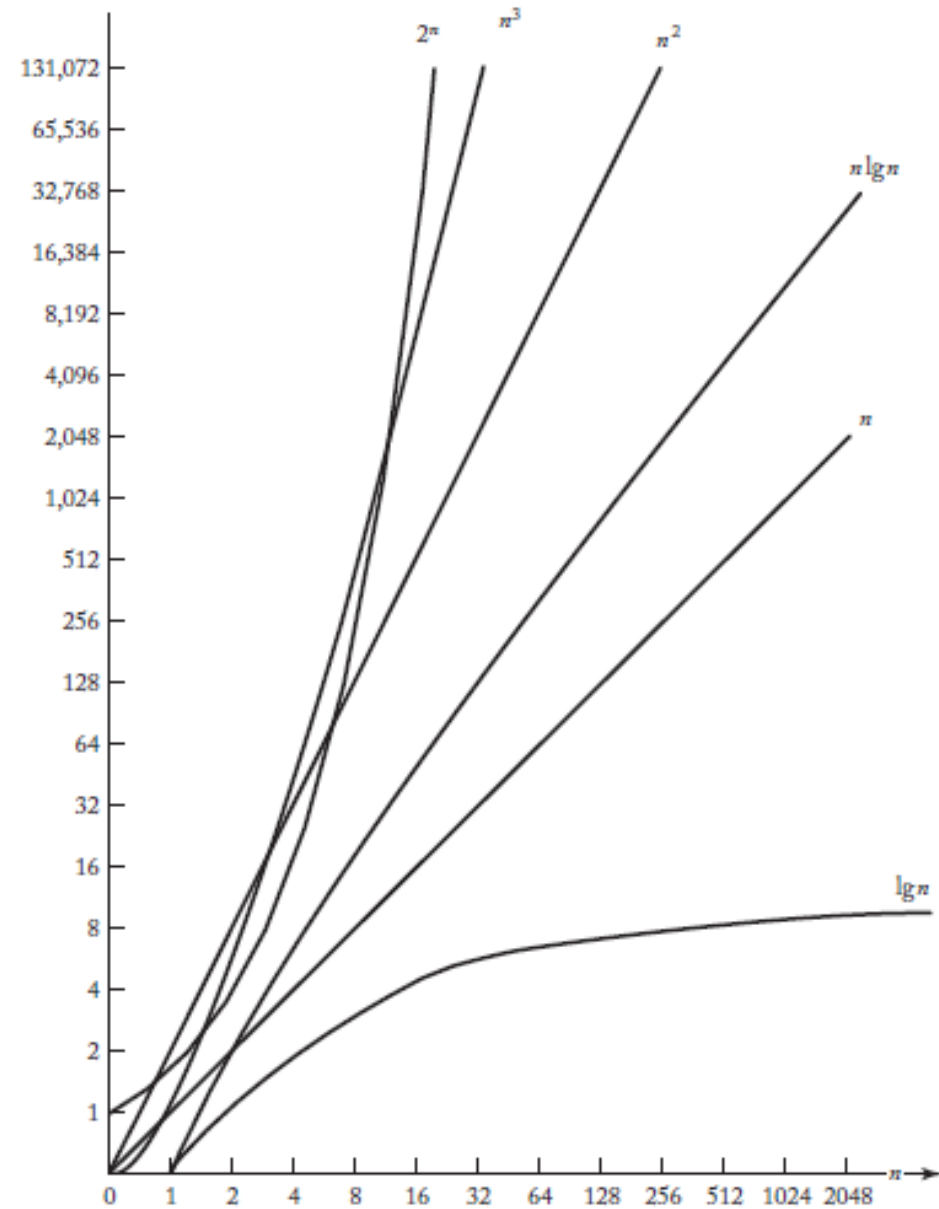
```
for (j = 0; j < n; j++ )  
{  
    statement;  
}
```

```
x = 0  
for (i = 1; i < n; i = i * 2)  
{  
    x++;  
}  
  
for (j = 1; j < x; j = j * 2 )  
{  
    statement;  
}
```

Time Complexities (Example)

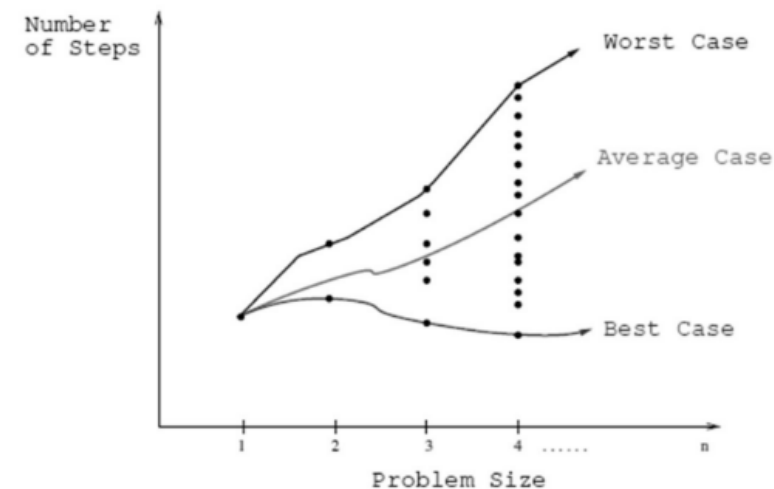
```
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j=j*2 )  
    {  
        statement;  
    }  
}
```

Growth Rate of Common Complexity Functions



Asymptotic Notation

- Asymptotic notation provides a way to describe the behaviour of algorithms as their input size grows. It gives a high-level understanding of an algorithm's efficiency by **abstracting away lower-order terms** and **constants** that have a minimal effect as the input size becomes large.
- Common Asymptotic Notations:
 - Big O (O)
 - Big Omega (Ω)
 - Big Theta (Θ)



$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$

Big O Notation

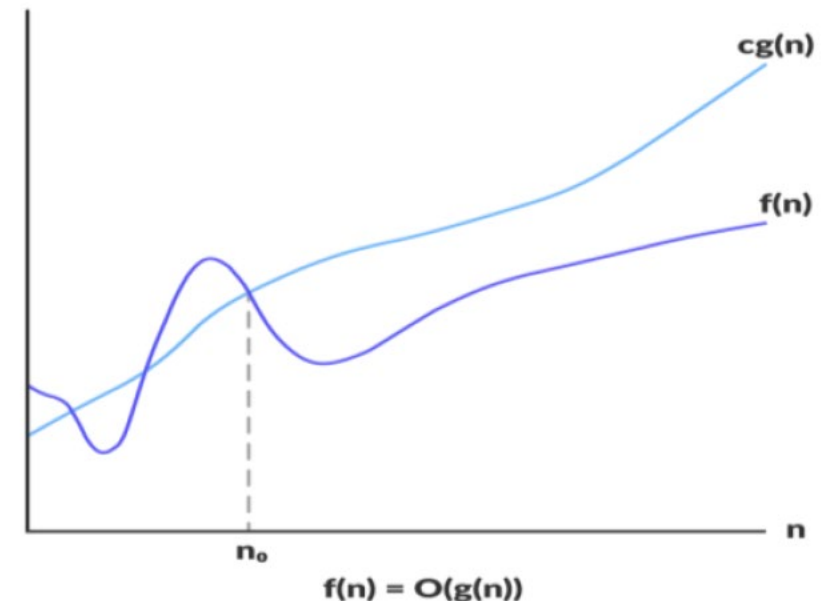
$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$

- Big O notation, often simply referred to as "Big O," describes the upper bound, or worst-case complexity, of an algorithm in terms of its input size. It provides a high-level understanding of how the runtime or space requirements of an algorithm grow as the input size increases.

Let $f(n)$ and $g(n)$ are two positive functions. A function $f(n)$ is said to be in $O(g(n))$ signified by $f(n) = O(g(n))$.

If $f(n)$ is bounded above by some positive constant multiple of $g(n)$ for large values of n , i.e. there exists some positive constant c and some nonnegative whole number n_0 with the end goal that

$f(n) \leq c g(n)$, when $n \geq n_0$.



Big Omega Notation

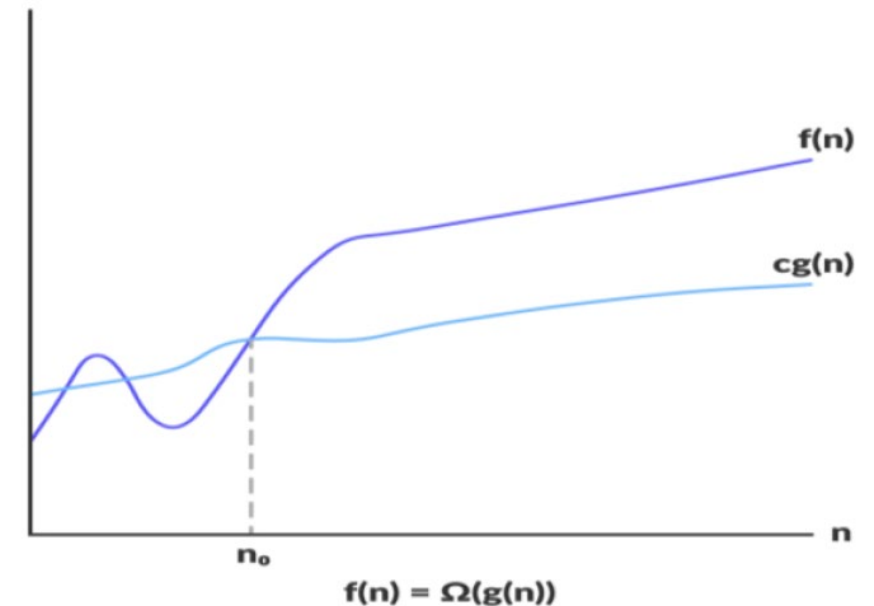
$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$

- Big Omega (Ω) notation is used in computer science to describe the lower bound, or best-case complexity, of an algorithm in terms of its input size. It explains the minimum number of operations an algorithm would need as the input size grows.

Let $f(n)$ and $g(n)$ are two positive functions. A function $f(n)$ is said to be in $\Omega(g(n))$ signified by $f(n) = \Omega(g(n))$.

If $f(n)$ is bounded lower by some positive constant multiple of $g(n)$ for large values of n , i.e. there exists some positive constant c and some nonnegative whole number n_0 with the end goal that

$$f(n) \geq c g(n), \text{ when } n \geq n_0.$$



Big Theta Notation

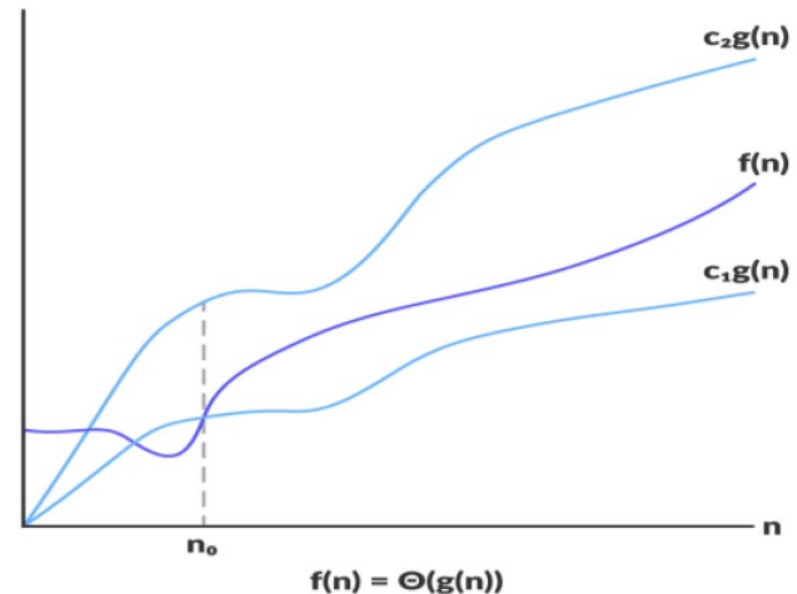
$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$

- Big Theta (Θ) notation describes the tight bound of an algorithm's complexity. It provides an understanding of an algorithm's performance, indicating that neither the upper bound (Big O) nor the lower bound (Big Omega) can be improved for that algorithm, hence giving a more precise characterization.

Let $f(n)$ and $g(n)$ are two positive functions.

A function $f(n)$ is said to be in $\Theta(g(n))$ signified by $f(n) = \Theta(g(n))$.

If $f(n)$ is limited above and below by some positive constant multiple of $g(n)$ for large values of n , i.e. there exists some positive constant c and some nonnegative whole number n_0 with the end goal that $c_1 g(n) \leq f(n) \leq c_2 g(n)$, when $n \geq n_0$.



Properties of Asymptotic Notations

- If $f(n)$ is $O(g(n))$, then $c * f(n)$ is $O(g(n))$
- If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$ then $f(n) = O(h(n))$
- If $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$, then $f(n) = \Theta(g(n))$
- If $f(n)$ is $\Theta(g(n))$ then $g(n)$ is $\Theta(f(n))$ – true only for Θ
- If $f(n) = O(g(n))$ then $g(n)$ is $\Omega(f(n))$ – true only for O and Ω
- If $f(n) = O(g(n))$ and $d(n) = O(h(n))$,
then $f(n) + d(n) = O(\max(g(n), h(n)))$
- If $f(n) = O(g(n))$ and $d(n) = O(h(n))$,
then $f(n) * d(n) = O(g(n) * h(n))$

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \dots < 2^n < 3^n < n^n$$